

⚡ JS async / await Cheat Sheet

Practical patterns and examples for writing robust, maintainable asynchronous code using async/await in JavaScript.

💡 Tip

Complete Dev Roadmap with SaaS Boilerplate at
thedevspace.io/

Basic Usage

```
1  async function fetchData() {  
2    const res = await fetch("/api");  
3    const data = await res.json();  
4    return data;  
5  }
```

- `async` marks a function that returns a Promise.
- `await` pauses execution until the Promise settles.
- Use try/catch for error handling inside async functions.

Error Handling

```
1 try {  
2   const data = await fetchData();  
3 } catch (err) {  
4   console.error(err);  
5 }
```

- Wrap `await` calls in `try/catch` or handle rejections upstream.
- Use timeouts or cancellation patterns for long-running tasks.

Parallelism

```
1 const [a, b] = await Promise.all([fetchA(),  
  fetchB()]);
```

- Use `Promise.all` for parallel awaits when tasks are independent.
- Avoid sequential awaits when parallelism is safe and desired.

Cancellation & AbortController

```
1  const controller = new AbortController();
2  fetch("/api", { signal: controller.signal })
3    .then((r) => r.json())
4    .catch((err) => {
5      if (err.name === "AbortError")
6        console.log("aborted");
7    });
8  // later
9  controller.abort();
```

- Use `AbortController` to cancel fetches and long-running tasks.
- Propagate abort signals to downstream work and clean up resources.
- Check `signal.aborted` in long-running loops or async tasks.

Timeouts

```
1 // timeout wrapper
2 function withTimeout(promise, ms) {
3   const timeout = new Promise((_, rej) =>
4     setTimeout(() => rej(new Error("Timeout")), ms)
5   );
6   return Promise.race([promise, timeout]);
7 }
```

- Wrap promises with a timeout to avoid hanging requests.
- Combine with `AbortController` where possible to cancel underlying work.
- Return clear error types (e.g., 'Timeout') for client side handling.

Promise Utilities

```
1 await Promise.all([a(), b()]);  
2 await Promise.allSettled([a(), b()]);  
3 await Promise.race([a(), b()]);  
4 await Promise.any([a(), b()]);
```

- `all` fails fast if any promise rejects.
- `allSettled` returns states for each promise (useful for best-effort parallelism).
- `race` resolves/rejects with the first settled promise
- `any` resolves with first fulfilled.
- Choose utility based on desired failure/fulfillment semantics.

Sequential vs Parallel Patterns

```
1 // sequential
2 for (const fn of tasks) await fn();
3
4 // parallel
5 await Promise.all(tasks.map((fn) => fn()));
```

- Use sequential when tasks depend on prior results; use parallel for independent tasks.
- Beware of over-parallelism (too many concurrent requests, aka throttle) when necessary.
- Use batching or concurrency limits for large task sets.

Retries & Backoff

```
1 // simple retry example
2 async function retry(fn, attempts = 3) {
3   for (let i = 0; i < attempts; i++) {
4     try {
5       return await fn();
6     } catch (e) {
7       if (i === attempts - 1) throw e;
8       await new Promise((r) => setTimeout(r, 100 *
9         2 ** i));
9     }
10   }
11 }
```

- Implement exponential backoff for transient errors (network, 5xx) and cap max delay.
- Be idempotent-aware, only retry safe or idempotent operations or use idempotency keys.
- Surface retryable vs terminal errors in API design.

Concurrency Control

```
1 // simple concurrency limiter
2 async function mapWithLimit(items, limit, fn) {
3   const results = [];
4   const executing = new Set();
5   for (const item of items) {
6     const p = Promise.resolve().then(() =>
fn(item));
7     results.push(p);
8     executing.add(p);
9     p.finally(() => executing.delete(p));
10    if (executing.size >= limit) await
Promise.race(executing);
11  }
12  return Promise.all(results);
13 }
```

- Limit concurrency for network-bound or CPU-bound tasks to avoid resource exhaustion.
- Use libraries (p-limit, p-queue) for robust features in production.
- Monitor resource usage and tune concurrency limits per environment.

Microtasks & Unhandled Rejections

```
1 Promise.resolve().then(() =>
  console.log("microtask"));
2 process.on("unhandledRejection", (err) => {
3   /* log and handle */
4 });
```

- Async/await schedules promise resolution in the microtask queue—understand ordering implications.
- Handle or surface rejections; use process-level handlers in Node to capture unhandledRejection.
- Prefer explicit try/catch around top-level awaits in async functions.

Resource Cleanup

```
1 // example: ensure stream is closed
2 try {
3   const res = await fetch("/stream");
4   // consume
5 } finally {
6   // close or abort resources
7   controller.abort();
8 }
```

- Use **finally** to clean up resources (streams, timers, sockets) even after errors.
- Tie cleanup to abort signals for coordinated shutdown.
- Avoid leaking timers or listeners by clearing them on teardown.

Best Practices

- Prefer `async/await` for readability but choose Promise utilities for coordination.
- Handle errors explicitly and design clear retry/cancellation semantics.
- Be mindful of concurrency and resource limits; test under load.
- Document API behavior for timeouts, retries, and idempotency.